

ARUNIMA SAHA

Generative AI Developer | AWS Data Engineer | LangChain | RAG | LangGraph
Bengaluru, India | arunimasaha11@gmail.com | +91 9187656193 | [LinkedIn](#) | [Credly](#) | [GitHub](#)

PROFESSIONAL SUMMARY

IT professional with 4 years of experience, transitioning from an **ETL Developer** to a **Data Engineer** and now to a **Generative AI Developer**, building production-focused **GenAI applications** using **LLMs, RAG, Graph RAG, LangChain, LangGraph, Neo4j, vector databases, AWS Cloud** and conversational AI frontends. Hands-on experience in designing and deploying **LLM-powered systems**, including a **Text-to-SQL chatbot**, a **Graph RAG conversational assistant** built on **Neo4j**, a **multi-agent travel planning system** using **LangGraph**, and a **serverless microservices backend** on **AWS**, all architected end-to-end.

Strong **AWS Cloud** foundation with **Lambda, ECS Fargate, ECR, EC2, API Gateway, RDS PostgreSQL, Secrets Manager, IAM, CloudWatch, and Redshift**, applied directly to **GenAI and data engineering production deployments**. Experienced in **prompt engineering**, including **few-shot prompting, chain-of-thought prompting, COSTAR**, and **schema injection**, as well as **retrieval optimization, Cypher query generation, two-stage LLM pipelines**, and **agentic workflow design** with **stateful graph execution**.

Solid **data engineering background** with experience in developing 40+ **PySpark ETL pipelines**, migrating **10M+ records** from **Teradata** to **Redshift**, orchestrating workflows using **Step Functions**, and performing **AWS Glue-based debugging**, bridging traditional **data engineering** with modern **AI application development**.

CORE TECHNICAL EXPERTISE

GenAI & LLM Engineering	Large Language Models (LLMs), GPT-4o, GPT-4o Mini, Prompt Engineering, Context Engineering, Few-Shot Prompting, Structured Prompting, COSTAR Framework, Text-to-SQL, Retrieval-Augmented Generation (RAG), Graph RAG, Token and Context Optimization, Two-Stage LLM Pipelines, Structured Outputs, Model Parameter Tuning
Agentic AI & Orchestration	LangChain, LangGraph, LCEL, GraphCypherQChain, Stateful Agent Workflows, Multi-Agent Systems, Tool and Function Calling, Agent Memory, Conditional Routing, State Schema Design, Human-in-the-Loop, LangSmith Tracing
RAG, Search & Knowledge Systems	RAG Architecture, Document Ingestion Pipelines, Chunking Strategies, Semantic Chunking, Embedding Models, OpenAI Embeddings, Sentence Transformers, Pinecone, Amazon OpenSearch, pgvector, Vector Search, Hybrid Search, Reranking, Retrieval Optimization, RAGAS Evaluation
Knowledge Graphs & Graph Databases	Neo4j, Cypher Query Language, Knowledge Graph Design, Node and Relationship Modeling, Multi-Hop Queries, Graph RAG, Graph-Based Retrieval
AWS Cloud & Application Deployment	AWS Lambda, Amazon EC2, Amazon ECS Fargate, Amazon ECR, Amazon EKS, Amazon API Gateway, Amazon S3, Amazon RDS for PostgreSQL, AWS Glue, AWS Step Functions, AWS Secrets Manager, AWS IAM, Amazon CloudWatch, Amazon Redshift, Docker
Backend & Data Engineering	Python, FastAPI, PySpark, SQL, ETL Pipeline Design, Data Warehousing, Dimensional Modeling, Data Migration, PostgreSQL, MySQL, REST APIs, Postman, DBever, pgAdmin, Git
AI Evaluation, Observability & Safety	LangSmith, CloudWatch Structured Logging, RAGAS, Faithfulness, Answer Relevance, Context Precision, Context Recall, Prompt Injection Defense, SQL Guardrails, Amazon Bedrock Guardrails, Responsible AI

PROFESSIONAL EXPERIENCE

Tata Consultancy Services

Sept 2022 – Present

Project 1: Multi-Agent Travel Planning System — LangGraph + GPT-4o

◆ Agentic Travel Planning & Booking System

Role: GenAI Developer

Technology Stack: Python, LangGraph, LangChain, GPT-4o, FastAPI, Chainlit, PostgreSQL, Duffel API, SerpAPI, Docker, AWS EKS

- Designed and developed an **agentic travel planning and booking system** using **LangGraph StateGraph**, orchestrating specialized workflow nodes for user-input processing, flight search, hotel recommendations, and travel-plan generation.
- Integrated the **Duffel API** for real-time flight search and **SerpAPI** for hotel and activity recommendations, with fallback handling to maintain application continuity during external API failures.
- Implemented **stateful conversation management** using LangGraph checkpoints, including **MemorySaver** and **PostgresSaver**, enabling session continuity through unique `thread_id` values.
- Developed **human-in-the-loop workflows** that allow users to review and select flights and hotels, modify travel preferences, validate budget constraints, and confirm bookings before final execution.
- Built a **FastAPI backend** and an interactive **Chainlit conversational interface**, enabling users to provide travel requirements and receive context-aware flight, hotel, and itinerary recommendations.
- Implemented persistent booking management using **PostgreSQL** and **JSONB-based storage**, supporting booking confirmation, reference generation, and retrieval of previously created travel bookings.
- Added structured handling for travel parameters including **source, destination, travel dates, passenger count, budget, currency, cabin class, and accommodation preferences**.
- Containerized the application using **Docker** and prepared cloud deployment configurations for **AWS EKS**, supporting scalable and cloud-native application deployment.

Project 2: Graph RAG Conversational Assistant — Neo4j + LangChain + GPT-4o

◆ Graph RAG Knowledge Graph Chatbot

Role: GenAI Developer **Stack:** Python, Neo4j, Cypher, LangChain, GraphCypherQChain, GPT-4o, LCEL, Chainlit, AWS EC2, FastAPI

- Designed and developed a **Graph RAG conversational assistant** over a **Neo4j knowledge graph**, enabling users to query interconnected travel-booking data through natural-language questions without requiring Cypher expertise.
- Modelled the travel domain using **six graph entity types**—Users, Bookings, Flights, Hotels, Cities, and Airlines—and relationship types including **HAS_BOOKING, INCLUDES_FLIGHT, INCLUDES_HOTEL, BOOKED_FLIGHT, and BOOKED_HOTEL** to support relationship-rich and multi-hop queries.
- Implemented **LangChain GraphCypherQChain** with a custom **schema-grounded Cypher generation prompt** and few-shot examples, enabling the LLM to translate user questions into executable Cypher queries based on the actual graph structure.
- Enabled **Cypher validation** and constrained generation to known schema properties and relationship types, improving the reliability of generated graph queries before database execution.
- Configured **enhanced schema extraction** to provide richer node-property and relationship context to the LLM, supporting more accurate query generation for ambiguous natural-language requests.
- Designed a **two-stage LLM pipeline**: the first stage generated and executed Cypher through GraphCypherQChain, while a second **LCEL formatting chain** transformed raw Neo4j results into clear, business-readable responses.
- Built an interactive **Chainlit conversational interface** that accepts user questions, invokes the Graph RAG pipeline, and returns formatted answers for counts, lists, bookings, airlines, hotels, and relationship-based travel queries.
- Deployed the end-to-end conversational application on **AWS EC2**, providing a cloud-hosted interface for natural-language exploration of Neo4j travel-booking data.

Project 3: AI-Based Text-to-SQL Chatbot — GPT-4o-mini + ECS Fargate + Lambda + RDS

◆ GenAI Text-to-SQL Conversational Assistant

Role: GenAI Developer

Stack: Python, OpenAI API, GPT-4o-mini, AWS Lambda, Chainlit, Docker, Amazon ECR, Amazon ECS Fargate, Amazon RDS for PostgreSQL, AWS Secrets Manager, AWS IAM, Amazon CloudWatch

- Developed an end-to-end **GenAI Text-to-SQL conversational assistant** that enables business users to query structured e-commerce data through natural-language questions and receive generated SQL, executed results, and business-readable responses.
- Architected a decoupled **serverless workflow** in which a **Chainlit** interface hosted on **Amazon ECS Fargate** invokes an LLM orchestration Lambda through a **Lambda Function URL**, while a separate query-executor Lambda handles database operations.
- Designed a **schema-aware prompting pipeline** for PostgreSQL using schema injection, strict table aliases, join rules, aggregation logic, case-insensitive filters, result limits, and SQL-only output enforcement.
- Implemented a multi-stage **GPT-4o-mini** workflow for intent classification, SQL generation, database execution, and conversational response synthesis; configured **temperature=0** for deterministic intent and SQL generation.
- Added an **intent-aware SQL validation layer** to verify that generated statements matched the detected operation before execution, with structured error handling across the orchestration and database layers.
- Built a reusable **AWS Lambda query-execution service** with PostgreSQL connection pooling, structured result serialization, transaction commit and rollback handling, and support for direct Lambda and API-compatible request payloads.

- Secured **OpenAI API credentials**, database connection details, model configuration, and Lambda references using **AWS Secrets Manager**, with cached secret retrieval and IAM-controlled service access.
- Implemented **structured CloudWatch logging** with UTC timestamps, request identifiers, generated-SQL visibility, execution status, and stack-trace-based error diagnostics for end-to-end troubleshooting.
- Containerized the LLM orchestration Lambda and Chainlit application using **Docker**, stored deployment images in **Amazon ECR**, and deployed the conversational frontend on **Amazon ECS Fargate**, with PostgreSQL hosted on **Amazon RDS**.
-

Data Engineering: Teradata Migration | ETL Pipelines | BISCUIT Batch System

- Led migration of **10M+ records** from on-premise Teradata to **AWS Redshift**; optimised distribution and sort keys, executed SQL-based transformations, and performed referential integrity validation post-migration.
- Built and maintained **40+ PySpark ETL pipelines** across SAP and JDE source systems using PostgreSQL for scalable data processing; monitored orchestration via **AWS Step Functions**; used **AWS Glue** for job failure debugging and root cause analysis.
- Resolved 15+ production incidents involving data discrepancies and pipeline failures; improved reporting accuracy by 25% through ETL optimization and data validation improvements.
- Communicated pipeline delays and system issues to stakeholders proactively; implemented contingency solutions to maintain SLA commitments during incidents.

CERTIFICATIONS & PROFESSIONAL DEVELOPMENT

Generative AI & Cloud AI

- Google Cloud Generative AI Leader (GAIL)
- AWS Generative AI Practitioner
- Extend Gemini Enterprise
- Vertex AI Search
- AI Applications — Build, Configure and Integrate

Data Analytics & Business Intelligence

- Google Data Analytics — Coursera
- IBM Data Analyst Professional Certificate — Coursera
- Microsoft Power BI — Udemy
- Microsoft Generative AI Essentials — LinkedIn Learning

EDUCATION

Master of Computer Applications (MCA) — Graphic Era University	2020–2022 CGPA: 8.2
Bachelor of Science — Information Technology (B.Sc. IT) — Graphic Era University	2017–2020 CGPA: 7.47

ATS KEYWORD INDEX

Generative AI | GenAI Engineer | LLM Engineer | AI Engineer | RAG Engineer | LangChain | LangGraph | LangSmith | AWS Bedrock | OpenAI | GPT-4o | Prompt Engineering | Context Engineering | RAG Pipeline | Agentic RAG | Graph RAG | Text-to-SQL | Vector Database | Pinecone | OpenSearch | pgvector | Neo4j | Knowledge Graph | Cypher | Multi-Agent System | Human-in-the-Loop | Agent Memory | Stateful Agents | Tool Calling | Conditional Routing | FastAPI | Python | PySpark | Pydantic | REST API | Docker | Lambda | ECS Fargate | ECR | API Gateway | RDS PostgreSQL | S3 | IAM | Secrets Manager | CloudWatch | AWS Redshift | Microservices | ETL | Data Engineering | Chainlit | RAGAS | Observability | Guardrails | Responsible AI | Data Migration | Step Functions | AWS Glue